KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY COIMBATORE – 641 407



ACADEMIC YEAR 2026 – 2027

LABORATORY MANUAL

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

U25CSG12 DATA STRUCTURES

KPRIET
Learn Beyond

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

ARASUR, COIMBATORE – 641407

DEPARTMENT OF COMPUER SCIENCE & ENGINEERING

Name :……

Roll number :……

Year & Semester :……

Class & Section :……

Course Code :……

Course Title :……

Certified that this is a bonafide work done by the above student during the academic year 2026 – 2027.

Faculty in-charge **HoD**

Certified that this is a bonafide work done by the above student during the academic year 2026 - 2027.

INTERNAL EXAMINER

VISION AND MISSION OF THE INSTITUTION

Vision

To become a premier institute of academic excellence by imparting technical, intellectual and professional skills to students for meeting the diverse needs of industry, society, the nation and the world at large.

Mission

- Commitment to offer value-based education and enhancement of practical skills.
- Continuous assessment of teaching and learning processes through scholarly activities.
- Enriching research and innovation activities in collaboration with industry and institutes of repute.
- Ensuring the academic processes to uphold culture, ethics and social responsibilities.

VISION AND MISSION OF THE DEPARTMENT

Vision

To be a premier centre for education, dissemination of knowledge and research in the frontier areas of computer science and engineering to serve the community with moral values.

Mission

- Provide holistic education incorporating the state-of-the-art technologies to produce successful professionals.
- Facilitate the students to pursue higher education and research in the areas related to Computer Science and Engineering.
- Promote strong collaborations with the industries and steer the students to nurture their interest in continuous learning to meet the changing needs of the society.

Program Educational Objectives

PEO 1: Acquire knowledge and skills on cutting edge technologies in the field of computer science and engineering.

PEO 2: Inculcate a passion for continuous learning through further studies and research in the field of computer science and engineering.

PEO 3: Develop innovative ideas upholding the rich value systems to solve the changing technological needs.

Program Outcomes (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment (WK1, WK5, and WK7)

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering

management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change (WK8) Knowledge and Attitude Profile (WK)

Program Specific Outcomes

PSO 1: Solve complex engineering problems by building integrated systems across various

domains of Computational Intelligence.

PSO 2: Apply technical skills to produce solutions that meet the requirements of the

emerging technologies in Computer Science and Engineering.

Course Outcomes (COs)

CO 1: Demonstrate the implementation of list-based linear data structures in data organization applications. (Apply)

CO2: Utilize stack and queue data structures to solve expression evaluation and process-handling problems. (Apply)

CO3: Apply searching, sorting, and hashing techniques for efficient data organization and retrieval. (Apply)

CO4: Employ tree structures and algorithms for hierarchical data representation and processing. (Apply)

CO5: Use graph algorithms for traversal, shortest path, and minimum spanning tree problems. (Apply)

Rubrics for Assessment

	Criteria	Excellent (4 Marks)	Good (3 Marks)	Adequate (2 Mark)	Inadequate (1 Mark)
A. Preparation & Observation	**Criteria #1** Ability to setup and conduct experiments	Able to develop contingency or alternative plans and anticipate problems during experiment.	Able to develop contingency or alternative plans.	Able to use theoretical framework, measurement techniques, testing apparatus or model.	Unable to identify theoretical framework, measurement techniques, testing apparatus or model.
	Criteria #2 Ability to take measurements / readings and present data	Able to formulate, controls and evaluate alternatives of the experiment. Able to evaluate data and relate to engineering phenomena for decision-making.	Able to evaluate data and relate to engineering phenomena for decision-making.	Able to apply constraint and assumption into the experimental design. Able to conduct experiment correctly and collect data.	Unable to discuss experimental processes and protocols.
B. Results & Interpretation	**Criteria #3** Ability to analyze the data theoretically and logically to conclude experimental results	Able to combine /organize more than one set of data, interpret data and make meaningful conclusion.	Able to evaluate or compare data and make meaningful conclusion.	Able to select and use and apply appropriate techniques or methods to analyze the data.	Unable to select and describe the techniques or methods of analyzing the data.
	Criteria #4 Ability to interpret and discuss any discrepancies between theoretical and experimental results	Able to verify and/or validate several sets of data and relates to engineering phenomena for decision making.	Able to verify and/or validate data and relate to engineering phenomena for decision making.	Able to identify and verify how results relate/differ from theory or previous results.	Unable to identify how results relate/differ from theory or previous results.
C. Viva-Voce	**Criteria #5** Demonstrate the ability to respond effectively to questions	Able to listen carefully and respond to questions appropriately; is able to explain and interpret results to the teacher.	Able to listen carefully and respond to questions appropriately.	Misunderstand the questions and does not respond appropriately to the teacher, or has some trouble in answering questions.	Unable to listen carefully to questions and does not provide an appropriate answer, or is unable to answer questions.

LIST OF EXPERIMENTS

BRANCH NAME: COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR: 2025-2026 (EVEN)

Course Code: U25CSG12 **SEMESTER: III**

Course Name: Data Structures

S. No	Name of the Experiments	Page No.	Marks (20)	Signature
1	Implement a music playlist where songs can be added, removed, or moved up/down dynamically using linked list			
2	Create a stack to manage “Back” and “Forward” operations in a browser, or evaluate arithmetic expressions			
3	Simulate customers arriving and being served at a bank counter using Queue ADT			
4	Implement bubble sort and Insertion sort			
5	Implement Shell sort algorithm			
6	Implement Linear search and Binary search algorithms			
7	Implement a hash function to Store and search books by ISBN or title efficiently			
8	Represent a file system as a tree and allow insertion/deletion of folders/files and implement tree traversal for the same			
9	Represent cities and roads as a graph and find shortest paths using Dijkstra's algorithm			
10	Implement prims algorithm to find minimum spanning tree of a given graph			

Exp. No:1	Implement a music playlist where songs can be added, removed, or moved up/down dynamically using linked list
Date:	

Aim

To implement a music playlist using a singly linked list that supports adding, removing, and dynamically moving songs up or down.

Algorithm

1. Start.
2. Define a structure Node containing the song name and a pointer to the next node.
3. To add a song, create a new node and insert it at the end of the linked list.
4. To remove a song, search for the node with the matching name, unlink it from the list, and free its memory.
5. To move a song up, locate the node and its predecessor, then adjust the links to swap their positions.
6. To move a song down, locate the node and its successor, then adjust the links to swap their positions.
7. Display the playlist by traversing the list from head to the last node.
8. Stop.

Program

```
/* Program 1: Music Playlist using Linked List (Add, Remove, Move Up/Down) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char song[50];
    struct Node *next;
};

struct Node *head = NULL;

void addSong(char *name) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    strcpy(newNode->song, name);
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Song '%s' added to playlist.\n", name);
}

void removeSong(char *name) {
    struct Node *temp = head, *prev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prev = temp;
        temp = temp->next;
```

```
    }
    if (temp == NULL) {
        printf("Song '%s' not found.\n", name);
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    free(temp);
    printf("Song '%s' removed from playlist.\n", name);
}

void moveUp(char *name) {
    struct Node *temp = head, *prev = NULL, *prevprev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prevprev = prev;
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL || prev == NULL) {
        printf("Cannot move '%s' up.\n", name);
        return;
    }
    prev->next = temp->next;
    temp->next = prev;
    if (prevprev == NULL)
        head = temp;
    else
        prevprev->next = temp;
    printf("Song '%s' moved up.\n", name);
}

void moveDown(char *name) {
    struct Node *temp = head, *prev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Cannot move '%s' down.\n", name);
        return;
    }
    struct Node *next = temp->next;
    temp->next = next->next;
    next->next = temp;
    if (prev == NULL)
        head = next;
    else
        prev->next = next;
    printf("Song '%s' moved down.\n", name);
}

void display() {
    struct Node *temp = head;
    printf("Playlist: ");
    if (temp == NULL) {
        printf("(empty)\n");
        return;
    }
```

```
    while (temp != NULL) {
        printf("%s", temp->song);
        if (temp->next != NULL) printf(" -> ");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    addSong("Song A");
    addSong("Song B");
    addSong("Song C");
    addSong("Song D");
    display();

    moveUp("Song C");
    display();

    moveDown("Song A");
    display();

    removeSong("Song B");
    display();

    return 0;
}
```

Output

```
Song 'Song A' added to playlist.
Song 'Song B' added to playlist.
Song 'Song C' added to playlist.
Song 'Song D' added to playlist.
Playlist: Song A -> Song B -> Song C -> Song D
Song 'Song C' moved up.
Playlist: Song A -> Song C -> Song B -> Song D
Song 'Song A' moved down.
Playlist: Song C -> Song A -> Song B -> Song D
Song 'Song B' removed from playlist.
Playlist: Song C -> Song A -> Song D
```

Result

The music playlist was successfully implemented using a singly linked list. Songs were added, removed, and moved up/down dynamically, and the output was verified successfully.

Experiment No. 1 — Viva Voce Questions

Music Playlist using Linked List (Add, Remove, Move Up/Down)

Q1. What is a linked list?

Q2. What is the difference between an array and a linked list?

Q3. What is the time complexity of inserting a node at the end of a singly linked list?

Q4. Why is a 'prev' pointer needed while deleting a node?

Q5. What happens if a deleted node is not freed?

Q6. How would moving a node up/down differ in a doubly linked list?

Q7. What is the time complexity of searching for a song by name?

Q8. Can this playlist implementation store duplicate song names?

Q9. What is the significance of the head pointer?

Q10. How can a playlist shuffle feature be implemented using this structure?

Ex. No. 1				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 2	Create a stack to manage “Back” and “Forward” operations in a browser, or evaluate arithmetic expressions
Date:	

Aim

To implement a browser navigation system using two stacks that simulates the *Back* and *Forward* operations by storing and retrieving visited web pages.

Algorithm

1. Create two stacks: **Back** and **Forward**.
2. Open the home page.
3. When a new page is visited:
    - Push the current page onto the Back stack.
    - Clear the Forward stack.
    - Make the new page the current page.
4. When **Back** is selected:
    - Push the current page onto the Forward stack.
    - Pop the previous page from the Back stack.
5. When **Forward** is selected:
    1. Push the current page onto the Back stack.
    2. Pop the next page from the Forward stack.
6. Display the current page.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char song[50];
    struct Node *next;
};

struct Node *head = NULL;

void addSong(char *name) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    strcpy(newNode->song, name);
    newNode->next = NULL;
```

```
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Song '%s' added to playlist.\n", name);
}

void removeSong(char *name) {
    struct Node *temp = head, *prev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Song '%s' not found.\n", name);
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    free(temp);
    printf("Song '%s' removed from playlist.\n", name);
}

void moveUp(char *name) {
    struct Node *temp = head, *prev = NULL, *prevprev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prevprev = prev;
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL || prev == NULL) {
        printf("Cannot move '%s' up.\n", name);
        return;
    }
    prev->next = temp->next;
    temp->next = prev;
    if (prevprev == NULL)
        head = temp;
    else
        prevprev->next = temp;
    printf("Song '%s' moved up.\n", name);
}

void moveDown(char *name) {
    struct Node *temp = head, *prev = NULL;
    while (temp != NULL && strcmp(temp->song, name) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Cannot move '%s' down.\n", name);
        return;
    }
    struct Node *next = temp->next;
```

```
    temp->next = next->next;
    next->next = temp;
    if (prev == NULL)
        head = next;
    else
        prev->next = next;
    printf("Song '%s' moved down.\n", name);
}

void display() {
    struct Node *temp = head;
    printf("Playlist: ");
    if (temp == NULL) {
        printf("(empty)\n");
        return;
    }
    while (temp != NULL) {
        printf("%s", temp->song);
        if (temp->next != NULL) printf(" -> ");
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    addSong("Song A");
    addSong("Song B");
    addSong("Song C");
    addSong("Song D");
    display();

    moveUp("Song C");
    display();

    moveDown("Song A");
    display();

    removeSong("Song B");
    display();

    return 0;
}
```

Output

```
Visited: Google
Visited: Wikipedia
Visited: YouTube

Current Page: Wikipedia
Current Page: Google
Current Page: Wikipedia
```

Result

Thus, the browser Back and Forward operations were implemented successfully using two stacks, and the program produced the expected output.

Experiment No. 2 — Viva Voce Questions

Q1. Why are two stacks used to implement the browser Back and Forward operations instead of a single stack?

Q2. Explain how the Back operation works using the Back and Forward stacks.

Q3. What happens to the Forward stack when a new webpage is visited after performing the Back operation? Why?

Q4. Why is the Stack data structure suitable for implementing browser history management?

Q5. What is the role of the currentPage variable in the program?

Q6. What is stack overflow and stack underflow? In which situations can they occur in this program?

Q7. What is the time complexity of the following operations in the browser history program?

Visit a new page

Back

Forward

Q8. How would you modify the program to store additional webpage information such as the URL, title, and timestamp instead of just the page name?

Q9. How can the browser history program be implemented using linked-list-based stacks instead of array-based stacks? What are the advantages?

Q10. Mention two real-world applications of the Stack data structure other than browser navigation.

Ex. No. 2				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 3	Simulate customers arriving and being served at a bank counter using Queue ADT
Date:	

Aim

To implement a Queue Abstract Data Type (Queue ADT) using an array in C and simulate customer arrivals and services at a bank counter using enqueue and dequeue operations.

Algorithm

Algorithm: Enqueue (Customer Arrival)

1. Start.
2. Check whether the queue is full.
3. If the queue is full, display "Queue Overflow".
4. Otherwise:
   - If the queue is empty, initialize front = 0.
   - Increment rear.
   - Insert the customer's token number at queue[rear].
5. Stop.

Algorithm: Dequeue (Customer Service)

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue Underflow".
4. Otherwise:
   - Remove the customer from queue[front].
   - If front == rear, set front = rear = -1.
   - Else increment front.
5. Stop.

Algorithm: Display Queue

1. Start.
2. Check whether the queue is empty.
3. If empty, display "No customers in the queue."
4. Otherwise, display all customers from front to rear.
5. Stop.

Program

```
/* Program 3: Simulate Customers Arriving and Being Served at a Bank Counter using Queue */
```

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;
void enqueue(int token)
{
   if(rear == MAX - 1)
   {
      printf("Queue Overflow\n");
   }
   else
   {
      if(front == -1)
         front = 0;
      rear++;
      queue[rear] = token;
      printf("Customer %d arrived.\n", token);
   }}

void dequeue()
{
   if(front == -1)
   {
      printf("No customers to serve.\n");
   }
   else
   {
      printf("Customer %d served.\n", queue[front]);
      if(front == rear)
         front = rear = -1;
      else
         front++;
   }
}
void display()
{
   int i;
   if(front == -1)
   {
      printf("Queue is empty.\n");
   }
   else
   {
      printf("Customers waiting: ");
      for(i = front; i <= rear; i++)
         printf("%d ", queue[i]);
      printf("\n");
   }
}
int main()
```

```
{
   int choice, token;
   do
   {
      printf("\n--- Bank Queue Simulation ---\n");
      printf("1. Customer Arrival\n");
      printf("2. Serve Customer\n");
      printf("3. Display Queue\n");
      printf("4. Exit\n");
      printf("Enter your choice: ");
      scanf("%d",&choice);
      switch(choice)
      {
         case 1:
            printf("Enter Customer Token: ");
            scanf("%d",&token);
            enqueue(token);
            break;
         case 2:
            dequeue();
           break;
         case 3:
            display();
            break;
         case 4:
            printf("Exiting...\n");
            break;
         default:
            printf("Invalid Choice!\n");
      }
   }
while (choice != 4);
   return 0;
}
```

Output

```
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter Customer Token: 20
Customer 20 arrived.

--- Bank Queue Simulation ---
1. Customer Arrival
```

```
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter Customer Token: 30
Customer 30 arrived.
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 3
Customers waiting: 20 30
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 2
Customer 20 served.
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 3
Customers waiting: 30
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 2
Customer 30 served.
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 3
Queue is empty.
--- Bank Queue Simulation ---
1. Customer Arrival
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice:4
```

Result

Thus, the simulation of customers arriving and being served at a bank counter using the Queue ADT was successfully implemented, and the program produced the expected output.

Experiment No. 3 — Viva Voce Questions

Simulation of Customers at a Bank Counter using Queue ADT

Q1. What is a queue?

Q2. What is the difference between a stack and a queue?

Q3. What are queue overflow and underflow?

Q4. What is a circular queue and why is it used?

Q5. What is the time complexity of enqueue and dequeue operations?

Q6. What are some real-life applications of queues?

Q7. What is a priority queue?

Q8. How can a queue be implemented using two stacks?

Q9. What is a deque?

Q10. Why might a linked list be preferred over an array for queue implementation?

Ex. No. 3				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 4	Implement bubble sort and Insertion sort
Date:	

Aim

To implement Bubble Sort and Insertion Sort algorithms in C to sort an array in ascending order.

Algorithm

Algorithm: Bubble Sort

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Repeat for i = 0 to n-2
   - Repeat for j = 0 to n-i-2
   - If A[j] > A[j+1], swap them.
5. Display the sorted array.
6. Stop.

Algorithm: Insertion Sort

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Repeat for i = 1 to n-1
   - Store A[i] in key.
   - Compare key with previous elements.
   - Shift larger elements one position to the right.
   - Insert key in the correct position.
5. Display the sorted array.
6. Stop.

Program

```
#include <stdio.h>
void bubbleSort(int arr[], int n)
{
  int i, j, temp;

  for(i = 0; i < n - 1; i++)
  {
    for(j = 0; j < n - i - 1; j++)
```

```
      {
         if(arr[j] > arr[j + 1])
         {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}

void insertionSort(int arr[], int n)
{
   int i, key, j;

   for(i = 1; i < n; i++)
   {
      key = arr[i];
      j = i - 1;

      while(j >= 0 && arr[j] > key)
      {
         arr[j + 1] = arr[j];
         j--;
      }

      arr[j + 1] = key;
   }
}

void display(int arr[], int n)
{
   int i;

   for(i = 0; i < n; i++)
      printf("%d ", arr[i]);

   printf("\n");
}

int main()
{
   int n, i, choice;
   int arr[100];

   printf("Enter number of elements: ");
   scanf("%d",&n);

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
```

```
    scanf("%d",&arr[i]);

  printf("\nChoose Sorting Method\n");
  printf("1. Bubble Sort\n");
  printf("2. Insertion Sort\n");
  printf("Enter your choice: ");
  scanf("%d",&choice);

  switch(choice)
  {
    case 1:
      bubbleSort(arr, n);
      printf("Sorted array using Bubble Sort:\n");
      display(arr, n);
      break;

    case 2:
      insertionSort(arr, n);
      printf("Sorted array using Insertion Sort:\n");
      display(arr, n);
      break;

    default:
      printf("Invalid Choice");
  }

  return 0;
}
```

Output

```
Enter number of elements: 5
Enter array elements:
6
2
8
4
1

Choose Sorting Method
1. Bubble Sort
2. Insertion Sort
Enter your choice: 1
Sorted array using Bubble Sort:
1 2 4 6 8
```

Result

Thus, the Bubble Sort and Insertion Sort algorithms were successfully implemented, and the program produced the expected sorted output.

Experiment No. 4 — Viva Voce Questions

Simulation of Customers at a Bank Counter using Queue ADT

Q1. What is the main difference between Bubble Sort and Insertion Sort?

Q2. What is the time complexity of Insertion Sort in the best, average, and worst cases?

Q3. What is the space complexity of Bubble Sort and Insertion Sort?

Q4. What is meant by an in-place sorting algorithm? Are Bubble Sort and Insertion Sort in-place?

Q5. What is a stable sorting algorithm? Are Bubble Sort and Insertion Sort stable? Explain.

Q6. Which sorting algorithm performs better for nearly sorted data? Why?

Q7. How can Bubble Sort be optimized to reduce unnecessary comparisons?

Q8. What is the significance of the "key" element in Insertion Sort?

Q9. Compare Bubble Sort and Insertion Sort based on the number of comparisons and swaps.

Q10. Under what circumstances would you prefer Bubble Sort over Insertion Sort, and vice versa?

Ex. No. 4				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 5	Implement Shell sort algorithm
Date:	

Aim

To implement the Shell Sort algorithm to sort a list of integers in ascending order and to analyze its output.

Algorithm

Step 1: Start.

Step 2: Read the number of elements `n`.

Step 3: Read the array elements `A[0...n-1]`.

Step 4: Initialize the gap value as `gap = n / 2`.

Step 5: Repeat the following steps while `gap > 0`:

1. For each element from index `gap` to `n - 1`:
    - Store the current element in a temporary variable `temp`.
    - Set `j = i`.
    - Compare `temp` with the element located `gap` positions before it.
    - While `j >= gap` and `A[j-gap] > temp`:
        - Move `A[j-gap]` to position `A[j]`.
        - Update `j = j - gap`.
    - Insert `temp` into its correct position at `A[j]`.
2. Reduce the gap by setting `gap = gap / 2`.

Step 6: Display the sorted array.

Step 7: Stop.

Program

```
#include <stdio.h>

void shellSort(int arr[], int n)
{
  int gap, i, j, temp;

  for (gap = n / 2; gap > 0; gap = gap / 2)
  {
```

```
      for (i = gap; i < n; i++)
      {
         temp = arr[i];

         for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
         {
            arr[j] = arr[j - gap];
         }

         arr[j] = temp;
      }
   }
}

int main()
{
   int n, i;
   int arr[100];

   printf("Enter the number of elements: ");
   scanf("%d", &n);

   printf("Enter the elements:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }

   shellSort(arr, n);

   printf("Sorted array:\n");

   for(i = 0; i < n; i++)
   {
      printf("%d ", arr[i]);
   }

   return 0;
}
```

Output

```
Enter the number of elements: 8

Enter the elements:
45 32 18 60 27 15 90 12
```

```
Sorted array:
12 15 18 27 32 45 60 90
```

Result

Thus, the Shell Sort algorithm was successfully implemented using the C programming language. The given list of elements was sorted in ascending order, and the expected output was obtained successfully.

Experiment No. 5 — Viva Voce Questions

Q1. What is Shell Sort?

Q2. What is the significance of the gap sequence?

Q3. What is the time complexity of shell sort?

Q4. How does shell sort improve upon plain insertion sort?

Q5. Is shell sort a stable sorting algorithm?

Q6. What is the space complexity of shell sort?

Q7. Name some commonly used gap sequences.

Q8. Why is shell sort called a generalization of insertion sort?

Q9. What is the worst-case time complexity of shell sort with the original gap sequence?

Q10. When is shell sort preferred over quicksort or merge sort?

Ex. No. 5				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 6	Implement Linear search and Binary search algorithms
Date:	

Aim

To implement Linear Search and Binary Search algorithms to search for a given element in an array and compare their outputs.

Algorithm

Algorithm 1: Linear Search

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the array elements A[0...n−1].

Step 4: Read the element key to be searched.

Step 5: Set i = 0.

Step 6: Repeat the following until i < n:

If A[i] == key, display "Element found at position i + 1" and stop.
Otherwise, increment i by 1.

Step 7: If the element is not found after checking all elements, display "Element not found."

Step 8: Stop.

Algorithm 2: Binary Search

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the sorted array elements A[0...n−1].

Step 4: Read the element key to be searched.

Step 5: Initialize:

low = 0
high = n − 1

Step 6: Repeat while low <= high:

Compute the middle index:
mid = (low + high) / 2
If A[mid] == key, display "Element found at position mid + 1" and stop.
If key < A[mid], set high = mid − 1.
Otherwise, set low = mid + 1.

Step 7: If low > high, display "Element not found."

Step 8: Stop.

Program

```
#include <stdio.h>
int linearSearch(int arr[], int n, int key)
{
   int i;
   for(i = 0; i < n; i++)
   {
      if(arr[i] == key)
         return i;
   }
   return -1;
}

int binarySearch(int arr[], int n, int key)
{
   int low = 0, high = n - 1, mid;

   while(low <= high)
   {
      mid = (low + high) / 2;

      if(arr[mid] == key)
         return mid;
      else if(key < arr[mid])
         high = mid - 1;
      else
         low = mid + 1;
   }

   return -1;
}
```

```
int main()
{
   int arr[100], n, i, key, choice, pos;

   printf("Enter the number of elements: ");
   scanf("%d", &n);

   printf("Enter the elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   printf("Enter the element to search: ");
   scanf("%d", &key);

   printf("\n1. Linear Search\n");
   printf("2. Binary Search\n");
   printf("Enter your choice: ");
   scanf("%d", &choice);

   if(choice == 1)
   {
      pos = linearSearch(arr, n, key);
   }
   else if(choice == 2)
   {
      pos = binarySearch(arr, n, key);
   }
   else
   {
      printf("Invalid Choice");
      return 0;
   }

   if(pos != -1)
      printf("Element found at position %d", pos + 1);
   else
      printf("Element not found");

   return 0;
}
```

Output

```
Enter the number of elements: 8

Enter the elements: 45 32 18 60 27 15 90 12
```

```
Enter the element to search: 27

Enter your choice: 1

Element found at position 5

Enter the number of elements: 8

Enter the elements: 12 15 18 27 32 45 60 90

Enter the element to search: 45

Enter your choice: 2

Element found at position 6
```

Result

Thus, the Linear Search and Binary Search algorithms were successfully implemented using the C programming language. The required element was searched successfully, and the expected output was obtained.

Experiment No. 6 — Viva Voce Questions

Q1. What is the time complexity of linear search?

Q2. What is the time complexity of binary search?

Q3. What precondition is required for binary search to work correctly?

Q4. Why is binary search faster than linear search?

Q5. Can binary search be applied efficiently to a linked list?

Q6. What is the space complexity of the iterative and recursive versions of binary search?

Q7. When would linear search be preferred over binary search?

Q8. What is the worst-case scenario for linear search?

Q9. How does binary search behave when duplicate elements are present?

Q10. What are some common variations of binary search?

Ex. No. 6				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 7	Implement a hash function to Store and search books by ISBN or title efficiently
Date:	

Aim

To implement a hash table in C for storing and searching books efficiently using ISBN as the key.

Algorithm

1. Start the program.
2. Define a structure Book containing ISBN and title.
3. Create a hash table of fixed size.
4. Implement a hash function: index = ISBN % TABLE_SIZE.
5. Insert books into the hash table using linear probing for collision handling.
6. Search for a book using ISBN.
7. Display the search result.
8. Stop the program.

Program

```
#include <stdio.h>
#include <string.h>

#define SIZE 10

// Structure for Book
struct Book {
   int isbn;
   char title[50];
   int occupied;
};

struct Book hashTable[SIZE];

// Hash Function
int hashFunction(int isbn) {
   return isbn % SIZE;
}

// Insert Book
void insertBook(int isbn, char title[]) {
   int index = hashFunction(isbn);
```

```
   while (hashTable[index].occupied == 1) {
      index = (index + 1) % SIZE; // Linear Probing
   }

   hashTable[index].isbn = isbn;
   strcpy(hashTable[index].title, title);
   hashTable[index].occupied = 1;

   printf("Book inserted successfully.\n");
}

// Search Book
void searchBook(int isbn) {
   int index = hashFunction(isbn);
   int start = index;

   while (hashTable[index].occupied != 0) {

      if (hashTable[index].isbn == isbn) {
         printf("\nBook Found!\n");
         printf("ISBN : %d\n", hashTable[index].isbn);
         printf("Title: %s\n", hashTable[index].title);
         return;
      }

      index = (index + 1) % SIZE;

      if (index == start)
         break;
   }

   printf("\nBook not found.\n");
}

// Display Hash Table
void display() {
   printf("\nHash Table\n");
   printf("-----------------------------\n");

   for (int i = 0; i < SIZE; i++) {
      if (hashTable[i].occupied)
         printf("%d --> ISBN:%d  Title:%s\n", i,
               hashTable[i].isbn,
               hashTable[i].title);
      else
         printf("%d --> Empty\n", i);
   }
}
```

```
int main() {

   int choice, isbn;
   char title[50];

   while (1) {

      printf("\n------ HASH TABLE MENU ------\n");
      printf("1. Insert Book\n");
      printf("2. Search Book\n");
      printf("3. Display\n");
      printf("4. Exit\n");

      printf("Enter choice: ");
      scanf("%d", &choice);

      switch (choice) {

      case 1:
         printf("Enter ISBN: ");
         scanf("%d", &isbn);

         printf("Enter Book Title: ");
         scanf(" %[^\n]", title);

         insertBook(isbn, title);
         break;

      case 2:
         printf("Enter ISBN to search: ");
         scanf("%d", &isbn);

         searchBook(isbn);
         break;

      case 3:
         display();
         break;

      case 4:
         return 0;

      default:
         printf("Invalid choice!\n");
      }
   }
}
```

Output

```
D:\DS\lab programs\EX 7.exe

------ HASH TABLE MENU ------
1. Insert Book
2. Search Book
3. Display
4. Exit
Enter choice: 1
Enter ISBN: 102
Enter Book Title: DS
Book inserted successfully.

------ HASH TABLE MENU ------
1. Insert Book
2. Search Book
3. Display
4. Exit
Enter choice: 1
Enter ISBN: 202
Enter Book Title: Java
Book inserted successfully.

------ HASH TABLE MENU ------
1. Insert Book
2. Search Book
3. Display
4. Exit
Enter choice: 2
Enter ISBN to search: 202

Book Found!
```

```
D:\DS\lab programs\EX 7.exe
ISBN : 202
Title: Java

------ HASH TABLE MENU ------
1. Insert Book
2. Search Book
3. Display
4. Exit
Enter choice: 3

Hash Table
------------------------------
0 --> Empty
1 --> Empty
2 --> ISBN:102  Title:DS
3 --> ISBN:202  Title:Java
4 --> Empty
5 --> Empty
6 --> Empty
7 --> Empty
8 --> Empty
9 --> Empty

------ HASH TABLE MENU ------
1. Insert Book
2. Search Book
3. Display
4. Exit
Enter choice:
```

Result

Thus the hash function program was successfully implemented to store and search books efficiently using ISBN.

Experiment No. 7 — Viva Voce Questions

Simulation of Customers at a Bank Counter using Queue ADT

Q1. What is hashing?

Q2. What is a collision in hashing?

Q3. What is chaining?

Q4. What is open addressing?

Q5. What is the average time complexity of search/insert operations in a well-designed hash table?

Q6. What is a load factor?

Q7. What makes a good hash function?

Q8. What is the worst-case time complexity of hashing with chaining?

Q9. Why is ISBN a suitable key for hashing?

Q10. What are some real-world applications of hashing?

Ex. No. 7				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 8	Represent a file system as a tree and allow insertion/deletion of folders/files and implement tree traversal for the same
Date:	

Aim

To implement a tree data structure in C to represent a file system and perform insertion, deletion, searching, and preorder traversal of folders and files.

Algorithm

Algorithm for File System Representation

1. Start the program.
2. Define a tree node structure containing:
   - Name of the file/folder.
   - Pointer to the first child.
   - Pointer to the next sibling.
3. Create the root directory named Root.
4. Display the menu with the following options:
   - Insert Folder/File
   - Delete Folder/File (Leaf Node)
   - Display File System
   - Exit
5. Read the user's choice.

Insertion Algorithm

1. Read the parent folder name.
2. Read the new file/folder name.
3. Search for the parent folder in the tree.
4. If the parent exists:
   - Create a new node.
   - If the parent has no child, insert it as the first child.
   - Otherwise, insert it as the last sibling of the existing children.
5. Otherwise, display "Parent Folder Not Found."

Deletion Algorithm

1. Read the name of the file/folder to be deleted.
2. Search for the specified node.
3. If the node is a leaf node:
   - Remove it from the tree.
   - Free the allocated memory.
4. Otherwise, display an appropriate message.
5. If the node is not found, display "Node Not Found."

Traversal Algorithm (Preorder)

1. Visit the root node.
2. Display the node name.

3. Recursively traverse the first child.
4. Recursively traverse the next sibling.
5. Continue until all nodes have been visited.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
   char name[50];
   struct Node *child;
   struct Node *sibling;
} Node;

// Function to create a new node
Node *createNode(char name[])
{
   Node *newNode;

   newNode = (Node *)malloc(sizeof(Node));

   strcpy(newNode->name, name);
   newNode->child = NULL;
   newNode->sibling = NULL;

   return newNode;
}

// Function to search for a node
Node *search(Node *root, char name[])
{
   Node *temp;

   if (root == NULL)
      return NULL;

   if (strcmp(root->name, name) == 0)
      return root;

   temp = search(root->child, name);

   if (temp != NULL)
      return temp;
```

```
    return search(root->sibling, name);
}

// Function to insert a file/folder
void insert(Node *root)
{
    char parentName[50];
    char childName[50];

    Node *parent;
    Node *temp;
    Node *newNode;

    printf("Enter Parent Folder: ");
    scanf("%s", parentName);

    printf("Enter New File/Folder: ");
    scanf("%s", childName);

    parent = search(root, parentName);

    if (parent == NULL)
    {
        printf("Parent Folder Not Found!\n");
        return;
    }

    newNode = createNode(childName);

    if (parent->child == NULL)
    {
        parent->child = newNode;
    }
    else
    {
        temp = parent->child;

        while (temp->sibling != NULL)
            temp = temp->sibling;

        temp->sibling = newNode;
    }

    printf("Inserted Successfully.\n");
}

// Delete only the immediate child of Root
void deleteNode(Node *root)
{
```

```
    char name[50];

    Node *temp;
    Node *prev;

    printf("Enter File/Folder Name to Delete: ");
    scanf("%s", name);

    temp = root->child;
    prev = NULL;

    while (temp != NULL)
    {
        if (strcmp(temp->name, name) == 0)
        {
            if (prev == NULL)
                root->child = temp->sibling;
            else
                prev->sibling = temp->sibling;

            free(temp);

            printf("Deleted Successfully.\n");
            return;
        }

        prev = temp;
        temp = temp->sibling;
    }

    printf("Node Not Found.\n");
}

// Preorder Traversal
void preorder(Node *root, int level)
{
    int i;

    if (root == NULL)
        return;

    for (i = 0; i < level; i++)
        printf("   ");

    printf("|-- %s\n", root->name);

    preorder(root->child, level + 1);
    preorder(root->sibling, level);
}
```

```
int main()
{
   Node *root;
   int choice;

   root = createNode("Root");

   while (1)
   {
      printf("\n===== FILE SYSTEM MENU =====\n");
      printf("1. Insert Folder/File\n");
      printf("2. Delete Folder (Root Child Only)\n");
      printf("3. Display File System\n");
      printf("4. Exit\n");

      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
      case 1:
         insert(root);
         break;

      case 2:
         deleteNode(root);
         break;

      case 3:
         printf("\nFile System Structure:\n");
         preorder(root, 0);
         break;

      case 4:
         printf("Exiting...\n");
         exit(0);

      default:
         printf("Invalid Choice!\n");
      }
   }

   return 0;
}
```

Output

```
D:\DS\lab programs\ex8.3.exe

===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
Enter your choice: 1
Enter Parent Folder: Root
Enter New File/Folder: kpr
Inserted Successfully.

===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
Enter your choice: 1
Enter Parent Folder: Root
Enter New File/Folder: k1
Inserted Successfully.

===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
Enter your choice: 3
```

```
===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
Enter your choice: 3

File System Structure:
|-- Root
    |-- kpr
    |-- k1

===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
Enter your choice: 2
Enter File/Folder Name to Delete: k1
Deleted Successfully.

===== FILE SYSTEM MENU =====
1. Insert Folder/File
2. Delete Folder (Root Child Only)
3. Display File System
4. Exit
```

Result

The file system was successfully represented using a tree data structure. The program performed insertion, deletion of leaf nodes, searching, and preorder traversal of folders and files correctly.

Experiment No. 8 — Viva Voce Questions

Simulation of Customers at a Bank Counter using Queue ADT

Q1. What is a queue?

Q1. Why is a tree a suitable structure to represent a file system?

Q2. What is the child-sibling representation of a tree?

Q3. What is a leaf node in this context?

Q4. What traversal technique is used to print the file system?

Q5. What happens when a folder containing children is deleted in this simple implementation?

Q6. What is the time complexity of searching for a file in this tree?

Q7. How does this tree differ from a binary tree?

Q8. What real-world operating system structures use trees?

Q9. How would you implement 'find file by name' across the whole tree?

Q10. What additional fields could be added to track file size or type?

Ex. No. 8				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 9	Represent cities and roads as a graph and find shortest paths using Dijkstra's algorithm
Date:	

Aim

To represent cities and roads as a graph and to find the shortest path from a source city to all other cities using Dijkstra's Algorithm.

Algorithm

1. Start.
2. Represent the graph as an adjacency matrix, where graph[i][j] stores the road distance between city i and city j (0 indicates no direct road).
3. Initialize a distance array dist[] with infinity for all cities except the source city (dist[source] = 0), and a visited array marked as false for all cities.
4. Repeat until all cities are visited: select the unvisited city u with the minimum distance value and mark it as visited.
5. For every neighboring city v of u, if dist[u] + weight(u, v) is less than dist[v], update dist[v] to this smaller value.
6. After all cities have been processed, dist[] contains the shortest distance from the source to every other city.
7. Stop.

Program

```
#include <stdio.h>
#define V 5
#define INF 9999

int minDistance(int dist[], int visited[]) {
   int min = INF, minIndex = -1, v;
   for (v = 0; v < V; v++) {
      if (!visited[v] && dist[v] <= min) {
         min = dist[v];
         minIndex = v;
      }
   }
   return minIndex;
}

void dijkstra(int graph[V][V], int src) {
   int dist[V];
   int visited[V];
```

```
    int i, count, u, v;

    for (i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;

    for (count = 0; count < V - 1; count++) {
        u = minDistance(dist, visited);
        visited[u] = 1;
        for (v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("City\tDistance from Source (City %d)\n", src);
    for (i = 0; i < V; i++)
        printf("%d\t%d\n", i, dist[i]);
}

int main() {
    /* Cities 0-4 connected by roads with distances (0 = no road) */
    int graph[V][V] = {
        {0, 4, 0, 0, 8},
        {4, 0, 8, 0, 0},
        {0, 8, 0, 7, 0},
        {0, 0, 7, 0, 9},
        {8, 0, 0, 9, 0}
    };

    dijkstra(graph, 0);

    return 0;
}
```

Output

```
City    Distance from Source (City 0)
0       0
1       4
2       12
3       17
4       8
```

Result

Dijkstra's algorithm was successfully implemented to find the shortest distance from the source city to all other cities in the graph. The computed shortest distances were verified to be correct.

Experiment No. 9 — Viva Voce Questions

Q1. What is Dijkstra's algorithm used for?

Q2. Does Dijkstra's algorithm work correctly with negative edge weights?

Q3. What is the time complexity of Dijkstra's algorithm using an adjacency matrix?

Q4. How can Dijkstra's algorithm be optimized?

Q5. What is a greedy algorithm?

Q6. What data structure is used to represent the graph in this program?

Q7. What is the difference between Dijkstra's algorithm and the Bellman-Ford algorithm?

Q8. What are some real-world applications of Dijkstra's algorithm?

Q9. What does the visited array represent in the algorithm?

Q10. Can Dijkstra's algorithm be used to find the shortest path between just two specific cities?

Ex. No. 9				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		

Exp. No: 10	Implement prims algorithm to find minimum spanning tree of a given graph
Date:	

Aim

To implement Prim's algorithm to find the Minimum Spanning Tree (MST) of a given weighted, connected graph.

Algorithm

1. Start.
2. Represent the graph using an adjacency matrix.
3. Initialize a key[] array with infinity for all vertices except the starting vertex (key[0] = 0), and an mstSet[] array marked as false for all vertices.
4. Repeat until all vertices are included in the MST: select the vertex u with the minimum key value that is not yet in mstSet, and include it in the MST.
5. For every vertex v adjacent to u, if edge weight graph[u][v] is less than key[v] and v is not already in mstSet, update key[v] and set parent[v] = u.
6. After all vertices are included, the parent[] array represents the edges forming the Minimum Spanning Tree.
7. Stop.

Program

```
#include <stdio.h>
#define V 5
#define INF 9999

int minKey(int key[], int mstSet[]) {
   int min = INF, minIndex = -1, v;
   for (v = 0; v < V; v++) {
      if (!mstSet[v] && key[v] < min) {
         min = key[v];
         minIndex = v;
      }
   }
   return minIndex;
}

void primMST(int graph[V][V]) {
   int parent[V];
   int key[V];
   int mstSet[V];
   int i, count, u, v;

   for (i = 0; i < V; i++) {
      key[i] = INF;
```

```
    mstSet[i] = 0;
  }

  key[0] = 0;
  parent[0] = -1;

  for (count = 0; count < V - 1; count++) {
    u = minKey(key, mstSet);
    mstSet[u] = 1;
    for (v = 0; v < V; v++) {
      if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
        parent[v] = u;
        key[v] = graph[u][v];
      }
    }
  }

  int totalWeight = 0;
  printf("Edge\tWeight\n");
  for (i = 1; i < V; i++) {
    printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
    totalWeight += graph[i][parent[i]];
  }
  printf("Total weight of MST = %d\n", totalWeight);
}

int main() {
  int graph[V][V] = {
    {0, 2, 0, 6, 0},
    {2, 0, 3, 8, 5},
    {0, 3, 0, 0, 7},
    {6, 8, 0, 0, 9},
    {0, 5, 7, 9, 0}
  };

  primMST(graph);

  return 0;
}
```

Output

```
Edge   Weight
0 - 1  2
1 - 2  3
0 - 3  6
1 - 4  5
Total weight of MST = 16
```

Result

Prim's algorithm was successfully implemented to construct the Minimum Spanning Tree of the given graph. The resulting MST edges and total weight were verified to be correct.

Experiment No. 10 — Viva Voce Questions

Q1. What is a Minimum Spanning Tree (MST)?

Q2. Is Prim's algorithm a greedy algorithm?

Q3. What is the time complexity of Prim's algorithm using an adjacency matrix?

Q4. How does Prim's algorithm differ from Kruskal's algorithm?

Q5. Does an MST always represent the shortest path between two vertices?

Q6. Can a graph have more than one Minimum Spanning Tree?

Q7. What is the role of the key[] array in Prim's algorithm?

Q8. Why must the graph be connected for an MST to exist?

Q9. What are some real-world applications of MST algorithms?

Q10. What data structure improves the efficiency of Prim's algorithm for sparse graphs?

Ex. No. 10				
S. No	**Rubrics**	**Max. Marks**	**Obtained Marks**	**Faculty sign with date**
1	**Preparation**	4		
2	**Observation**	4		
3	**Results**	4		
4	**Interpretation**	4		
5	**Viva-voce**	4		
Total		**20**		